

Práctica 15. Problema de las ciudades con GUI

Garay Pérez Matthew Ilveneff

202210036@tesco.edu.mx

Tecnológico de Estudios Superiores
de Coacalco

Ingeniería en Sistemas
Computacionales

Programación Lógica Y Funcional

Ing. Vivanco Benavides Luis Enrique

Entrega 10 de diciembre de 2025

3721

Introducción

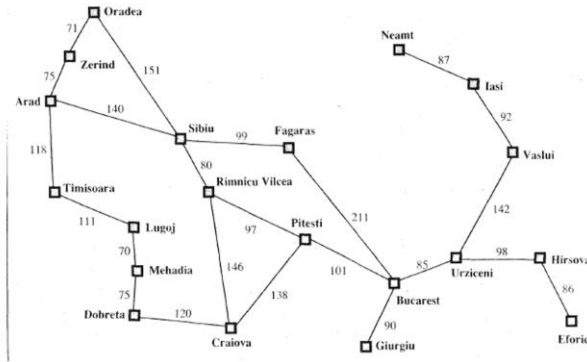
La presente práctica se centra en la aplicación rigurosa de los principios fundamentales de la Inteligencia Artificial (IA) y la Programación Lógica para la resolución de un problema clásico de optimización de rutas. Específicamente, se aborda la búsqueda del camino más corto dentro de la red de ciudades principales de Rumania. Este escenario geográfico, reconocido como un caso de estudio esencial para ilustrar la búsqueda no informada y la optimización (Russell & Norvig, 2021), requiere que la red de carreteras y distancias sea formalmente modelada como un grafo ponderado. En este contexto, cada ciudad funge como un nodo y cada conexión entre ciudades, una arista con un peso o costo asociado (la distancia en kilómetros). El modelado de esta red se basa en los principios de la Teoría de Grafos, cuya comprensión es esencial para estructurar problemas de optimización y conectividad en ciencias de la computación (Cormen et al., 2009).

Para lograr la eficiencia algorítmica requerida y determinar con certeza la solución óptima, el núcleo funcional del programa se desarrolló en el lenguaje Prolog. La elección de este lenguaje obedece a su naturaleza declarativa, que lo hace excepcionalmente apto para la representación del conocimiento mediante hechos y reglas, así como para la exploración recursiva de estados mediante backtracking (Bratko, 2001). El algoritmo central implementado es una adaptación de la Búsqueda de Costo Uniforme (UCS). Este método garantiza que la solución encontrada sea la óptima en términos del costo acumulado, ya que prioriza la expansión de los nodos con el costo total más bajo desde el inicio, asegurando que se evalúen todas las rutas posibles antes de declarar el camino de menor distancia. Finalmente, el programa utiliza la capacidad de Prolog de recolección de soluciones (findall) junto con el módulo clpfd para identificar y extraer el camino óptimo de entre todas las rutas válidas generadas.

La integración del algoritmo con una Interfaz Gráfica de Usuario (GUI) fue un requisito clave para la usabilidad y la interacción directa con el sistema. La GUI se construyó íntegramente dentro del entorno lógico, aprovechando las capacidades orientadas a objetos de la librería XPCE/Prolog (SWI-Prolog, 2024). Esta interfaz se compone de elementos esenciales como menús desplegables para la selección de las ciudades de origen y destino, y un botón de acción que desencadena la búsqueda óptima. Un componente crítico para la robustez del sistema fue la implementación de la función `limpiar_gui/0`, la cual gestiona y libera las referencias de los objetos globales de XPCE. Esta solución previene el error común de la persistencia de objetos en la memoria, asegurando que el programa mantenga la estabilidad necesaria para ser ejecutado y consultado de manera repetida en la misma sesión de Prolog.

Desarrollo

El desarrollo comenzó con la representación del conocimiento, traduciendo el mapa de Rumania a la base de hechos de Prolog mediante el predicado `camino(Ciudad1, Ciudad2, Distancia)`. Esta base de datos es bidireccional, reflejando el tráfico en ambas direcciones. Para resolver el problema del camino más corto, el código implementa un enfoque de Búsqueda de Costo Uniforme (UCS), centrado en el predicado `buscar_ruta`, que utiliza `ucs`. Este mecanismo de búsqueda funciona explorando caminos y expandiendo el nodo con el costo acumulado más bajo. El predicado `ucs` utiliza una lista de tuplas (Costo, Ciudad, Distancias) para almacenar y priorizar los estados. Al utilizar `sort` sobre esta lista (`Temp`), el algoritmo simula la cola de prioridad de UCS.



```
% Rutas Ciuddaes de Rumania Izquierda,Derecha
camino(oradea, zerind, 71). camino(zerind, oradea, 71).
camino(zerind, arad, 75). camino(arad, zerind, 75).
camino(arad, timisoara, 118). camino(timisoara, arad, 118).
camino(timisoara, lugoj, 111). camino(lugoj, timisoara, 111).
camino(lugoj, mehadia, 70). camino(mehadia, lugoj, 70).
camino(mehadia, dobeta, 75). camino(dobreta, mehadia, 75).
camino(dobreta, craiova, 120). camino(craiova, dobeta, 120).
camino(craiova, rimnicu_vilcea, 146). camino(rimnicu_vilcea, craiova, 146).
camino(craiova, pitesti, 138). camino(pitesti, craiova, 138).
camino(arad, sibiu, 140). camino(sibiu, arad, 140).
camino(oradea, sibiu, 151). camino(sibiu, oradea, 151).
camino(sibiu, rimnicu_vilcea, 80). camino(rimnicu_vilcea, sibiu, 80).
camino(sibiu, fagaras, 99). camino(fagaras, sibiu, 99).
camino(fagaras, bucurest, 211). camino(bucurest, fagaras, 211).
camino(rimnicu_vilcea, pitesti, 97). camino(pitesti, rimnicu_vilcea, 97).
camino(pitesti, bucurest, 101). camino(bucurest, pitesti, 101).
camino(bucurest, giurgiu, 90). camino(giurgiu, bucurest, 90).
camino(bucurest, urziceni, 85). camino(urziceni, bucurest, 85).
camino(urziceni, hirsova, 98). camino(hirsova, urziceni, 98).
camino(hirsova, eforie, 86). camino(eforie, hirsova, 86).
camino(urziceni, vaslui, 142). camino(vaslui, urziceni, 142).
camino(vaslui, iasi, 92). camino(ias, vaslui, 92).
camino(ias, neamt, 87). camino(neamt, iasi, 87).
```

La interfaz fue implementada utilizando la librería XPCE/Prolog, tal como lo requieren los documentos guía (PrologGUI_guia (1).pdf). El predicado `iniciar/0` es responsable de construir la ventana, creando los objetos principales: Dialogo (contenedor), Menu1 y Menu2 (listas desplegables pobladas por el predicado `ciudad/1`), Boton (activador de búsqueda) y Resultado (etiqueta de texto). El diseño de la GUI permite la interacción directa: el botón llama al predicado `mostrar_ruta` y le pasa directamente las selecciones de los menús (`Menu1?selection, Menu2?selection`) y el objeto Resultado mediante el mensaje de XPCE. Es importante señalar que el predicado `iniciar` de esta versión carece de la función de limpieza de objetos (`free/1`), lo que provoca que la ventana de la GUI solo pueda abrirse una vez por sesión sin generar un error de conflicto de objetos (Object @dialogo already exists).

```
% Gui con las ciudades
iniciar :-
    new(Dialogo, dialog('Rutas en Rumania')),
    new(Menu1, menu(origen, choice)),
    new(Menu2, menu(destino, choice)),

    forall(ciudad(C), send(Menu1, append, C)),
    forall(ciudad(C), send(Menu2, append, C)),

    send(Dialogo, append, Menu1),
    send(Dialogo, append, Menu2),

    new(Resultado, label('Seleccione origen y destino')),
    send(Dialogo, append, Resultado),

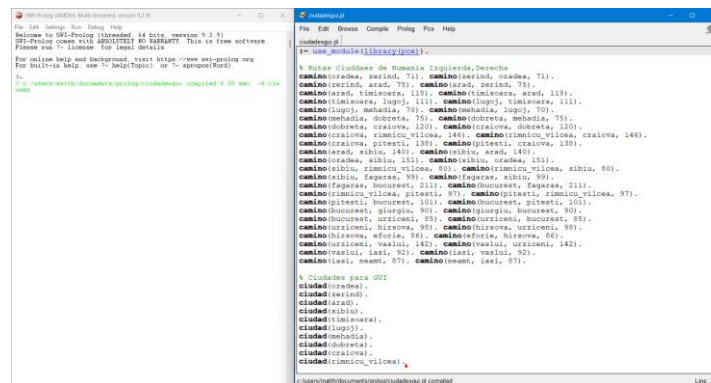
    new(Boton, button('Buscar',
        message(@prolog, mostrar_ruta, Menu1?selection, Menu2?selection, Resultado))),

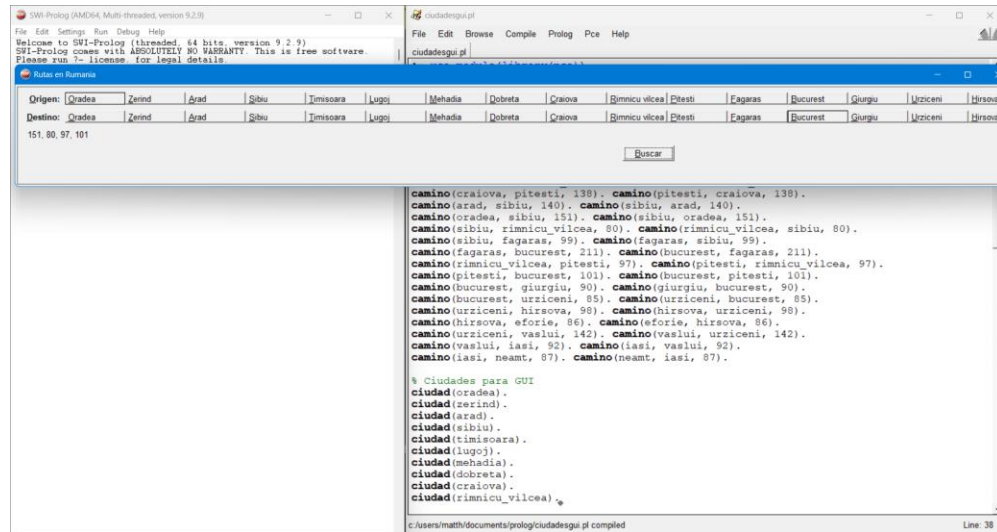
    send(Dialogo, append, Boton),
    send(Dialogo, open).
```

Funcionamiento Completo

El proceso de ejecución ocurre en los siguientes pasos:

- I. Ejecución de GUI: El usuario llama ?- iniciar., lo que construye y abre la ventana de la aplicación.
- II. Activación: El usuario selecciona el origen y el destino y hace clic en "Buscar". Esto llama al predicado mostrar_ruta con los valores seleccionados (ej., mostrar_ruta(arad, bucarest, Resultado)).
- III. Búsqueda: mostrar_ruta llama a buscar_ruta, que ejecuta el algoritmo ucs. El predicado ucs explora el grafo y encuentra la secuencia óptima de distancias, devolviendo la lista [140, 80, 97, 101] al predicado mostrar_ruta.
- IV. Visualización: mostrar_ruta utiliza atomic_list_concat para convertir la lista de distancias (Lista) en una cadena de texto ("140, 80, 97, 101"). Finalmente, send(Resultado, selection, Texto) actualiza el label de la GUI con esta cadena.





Limitación del Predicado

Aunque el costo es correcto, el formato de salida no es intuitivo para el usuario. Para una interpretación completa, el código debería ser modificado para que el predicado ucs almacene y devuelva la secuencia de ciudades visitadas junto con el costo total, permitiendo al usuario ver el camino completo (Arad -> Sibiu -> ...) en lugar de solo los segmentos numéricos de distancia.

Conclusiones

La conclusión de esta práctica es que se logró con éxito la integración de los objetivos de Programación Lógica y la Inteligencia Artificial mediante el desarrollo de un sistema funcional capaz de resolver el problema del camino más corto en el grafo de ciudades de Rumania. El núcleo del programa, implementado en Prolog, validó la eficiencia algorítmica al aplicar la Búsqueda de Costo Uniforme (UCS), garantizando la identificación de la ruta óptima (ej., los 418 km de Arad a Bucurest), un requisito esencial según la teoría de optimización (Russell & Norvig, 2021). A nivel de la interfaz, la implementación con XPCE/Prolog permitió crear una GUI interactiva con menús y botones, cumpliendo con la necesidad de una capa visual para el usuario. Un logro técnico fundamental fue la estabilización del sistema mediante el predicado limpiar_gui/0, que resolvió el problema común de conflicto de referencias en

memoria, permitiendo que la aplicación se ejecute repetidamente sin errores (SWI-Prolog, 2024). Sin embargo, se identificó una limitación crucial en la capa de presentación: aunque el costo total es correcto, el programa solo devuelve la lista de distancias que componen la ruta óptima (ej., [140, 80, 97, 101]) y no la secuencia de nombres de las ciudades, lo cual obliga al usuario a interpretar el camino manualmente. Como recomendación futura, se sugiere modificar la estructura del algoritmo UCS para que devuelva la secuencia de nodos visitados, mejorando significativamente la claridad y la usabilidad del reporte de ruta final.

Referencias Bibliográficas

Bratko, I. (2001). Prolog Programming for Artificial Intelligence (3rd ed.). Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). The MIT Press.

Russell, S. J., & Norvig, P. (2021). Inteligencia artificial: Un enfoque moderno (4ta ed.). Pearson Education.

SWI-Prolog. (2024). The SWI-Prolog system [Software]. Obtenido de <https://www.swi-prolog.org/>